CS 429/529 Assignment 1

Youssef Amin and Valerie Barker

Overview

This report describes the implementation and analysis of Adaline and logistic regression for supervised classification tasks. Both models are reformulated by absorbing the bias term into the weight matrix by adding a column of ones, yielding an equivalent but simplified representation. The performance of Adaline and logistic regression is examined on binary subsets of the Iris and Wine datasets by analyzing the convergence behavior of their respective loss functions under chosen training parameters. To address multiclass classification, a perceptron-based one-vs-rest approach is developed and applied to the full Iris dataset. Finally, stochastic gradient descent (SGD) and mini-batch SGD are implemented for logistic regression, and their loss convergence behavior is compared with standard gradient descent.

1 Task 1

In the standard Adaline and logistic regression models, the net input is computed as a weighted sum of the input features plus a bias term,

$$z = \mathbf{w}^T \mathbf{x} + b.$$

The bias term can be absorbed into the weight vector by appending a column of ones to the input matrix. The bias is then represented as an extra weight for each vector. After this transformation, the net input can be written as

$$z = \tilde{\mathbf{w}}^T \tilde{\mathbf{x}},$$

where $\tilde{\mathbf{x}} = [x_1, x_2, \ldots, x_n, 1]$ and $\tilde{\mathbf{w}} = [w_1, w_2, \ldots, w_n, b]$.

This reformulation is mathematically equivalent to the original model because the constant feature ensures that the bias contributes the same offset to every input. The learning behavior and decision boundaries remain unchanged, while the implementation is simplified by treating the bias as part of the weight vector.

2 Task 2

In this task, Adaline and logistic regression were compared using the Iris and Wine datasets. The comparison was based on how the loss changed over training epochs.

2.1 Iris Dataset

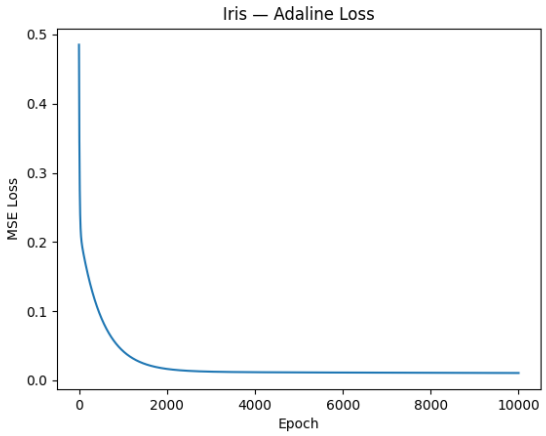

Figure 1: Adaline loss convergence on the Iris dataset

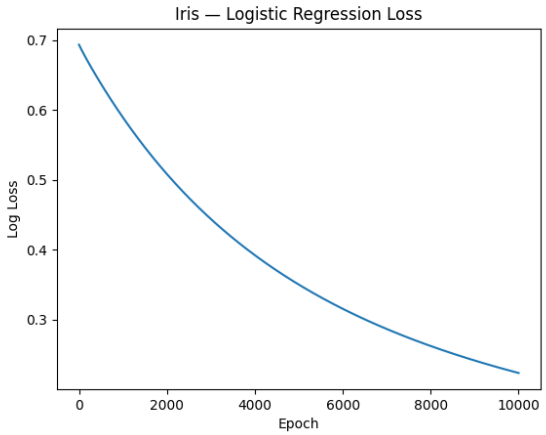

Figure 2: Logistic regression loss convergence on the Iris dataset

For the Iris dataset, Adaline converges very quickly. The mean squared error drops sharply during the first part of training and then levels off at a very small value. This suggests that the selected Iris classes are close to being linearly separable. Logistic regression also converges, but it does so more slowly, with the log loss decreasing gradually over time.

2.2 Wine Dataset

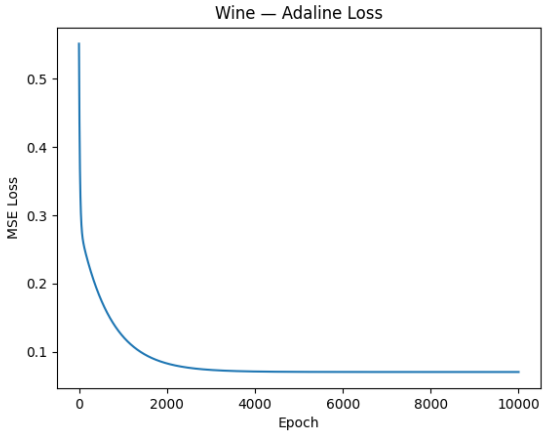

Figure 3: Adaline loss convergence on the Wine dataset

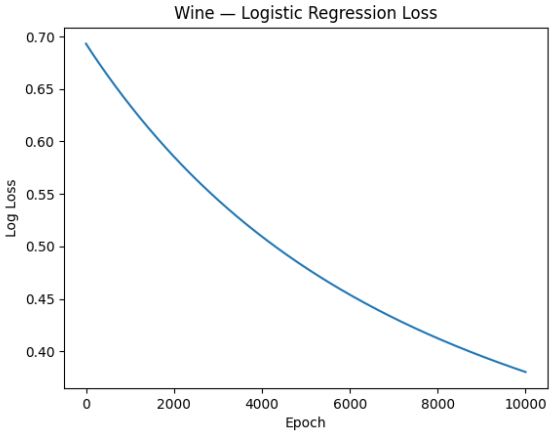

Figure 4: Logistic regression loss convergence on the Wine dataset

The Wine dataset shows similar behavior. Adaline again converges faster, with the loss decreasing quickly before stabilizing. Logistic regression converges more slowly and shows a smoother decrease in loss across epochs. Both models reach stable loss values, but the Wine dataset appears to be more difficult than the Iris dataset based on the final loss values.

Overall, Adaline converges faster than logistic regression on both datasets, while logistic regression shows a more gradual learning process.

3 Task 3

Adaline and perceptron learning algorithms are limited to binary classification, while the full Iris dataset contains three classes: setosa, versicolor, and virginica. To address this limitation, a one-vs-rest multiclass classification approach was used.

In this approach, three separate perceptrons are trained, one for each Iris class. Each perceptron is trained to recognize its assigned class as the positive label, while the remaining two classes are treated as negative. During prediction, all three perceptrons compute their net input values for a given sample, and the class corresponding to the perceptron with the highest net input is selected as the predicted class.

Figure 5 shows the number of misclassifications per epoch for each perceptron during training. The perceptron trained to identify Iris-setosa converges quickly and reaches zero classification errors, indicating that this class is easily separable from the others. The perceptrons for Iris-versicolor and Iris-virginica converge more slowly and stabilize at higher error values, reflecting the greater overlap between these two classes. Overall, the results show that the one-vs-rest approach is effective, though its performance depends on how well each class can be separated from the rest.

The following code snippet shows the core logic used to perform multiclass prediction by comparing the outputs of multiple perceptrons.

```
def predict_multiclass(X, models, class_names):
    scores = np.vstack([m.net_input(X) for m in models])
    winner = np.argmax(scores, axis=0)
    return class_names[winner]
```

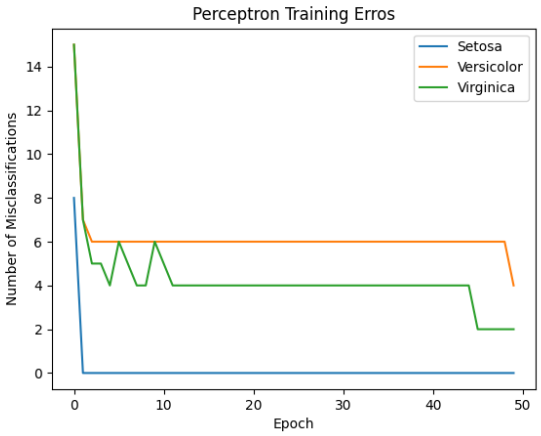

Figure 5: Number of misclassifications per epoch for each perceptron during multiclass Iris training

4 Task 4

In this task, stochastic gradient descent (SGD) and mini-batch SGD were implemented for logistic regression and compared based on their loss convergence behavior during training.

With SGD, the model updates its weights after processing each individual training example. As shown in Figure 6, the loss decreases very rapidly during the early stages of training but exhibits significant noise. This behavior is expected, since each update is based on a single data point, causing high variance in the loss values across epochs.

Mini-batch SGD updates the model weights using small batches of samples at a time. Figure 7 shows a much smoother loss curve compared to standard SGD. This smoother behavior occurs because the loss at each epoch is computed by averaging the losses across all mini-batches, which reduces variance introduced by individual samples. Although mini-batch SGD converges more slowly than SGD in the early stages, it provides a more stable training process.

Overall, SGD converges faster but produces a noisy loss curve due to per-sample updates, while mini-batch SGD offers a balance between convergence speed and stability by averaging updates over batches.

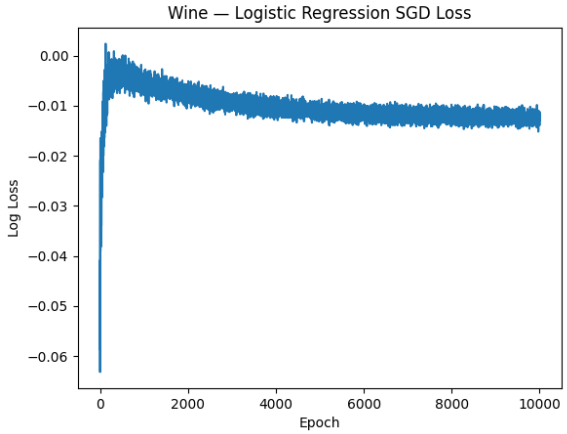

Figure 6: Logistic regression loss convergence using stochastic gradient descent on the Iris dataset

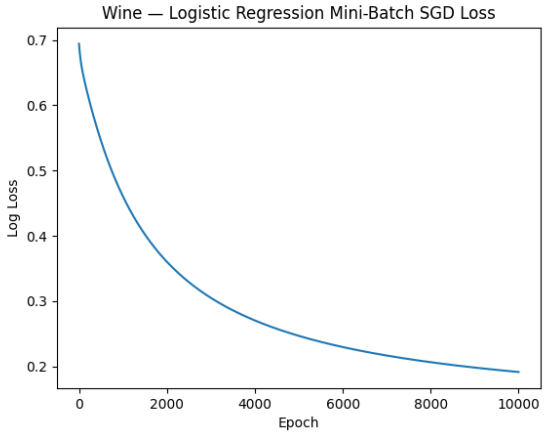

Figure 7: Logistic regression loss convergence using mini-batch SGD on the Iris dataset

Project Repository

The project repository is organized to separate model implementations, data handling, and experiment scripts. The purpose of each file is summarized below.

- `ModifiedAdalineGD.py` - Implementation of the Adaline algorithm with the bias absorbed into the weight vector.
- `ModifiedLogisticRegressionGD.py` - Implementation of logistic regression using batch gradient descent with bias absorbed into the weights.
- `logisticRegressionSGD.py` - Logistic regression implementation using stochastic gradient descent.
- `logisticMiniBatchSGD.py` - Logistic regression implementation using mini-batch stochastic gradient descent with averaged batch loss.
- `cerberus.py` - One-vs-rest perceptron implementation used for multiclass classification on the full Iris dataset.
- `datasetImport.py` - Functions for loading and preprocessing the Iris and Wine datasets.
- `normalization_utils.py` - Utility functions for normalizing input data.
- `compare_losses.py` - Main script used to train all models and generate the plots included in the report.
- `TaskOneTest.py` - Simple test script used to verify the correctness of the bias-absorbed models.
- `images/` - Directory containing all figures generated by the experiments and referenced in the report.

Conclusion

In this assignment, Adaline and logistic regression were implemented and analyzed across multiple classification settings. By absorbing the bias term into the weight vector, both models were simplified without changing their behavior. Experiments on the Iris and Wine datasets showed that Adaline generally converges faster, while logistic regression converges more gradually. A one-vs-rest perceptron approach was successfully used to extend binary classification to multiclass classification on the full Iris dataset, with performance depending on how well each class could be separated. Finally, comparisons between standard gradient descent, SGD, and mini-batch SGD highlighted the tradeoff between convergence speed and stability, with mini-batch SGD providing smoother and more stable training behavior. Overall, the results demonstrate how different learning algorithms and optimization strategies affect convergence and performance in practice.